New GPU automatic kernel fusion species

Oscar Amoros 1 (oscar.amoros.huguet@upc.edu), Antonio J. Peña 1,2 (antonio.pena@bsc.es)

1 Universitat Politècnica de Catalunya, 2 Barcelona Supercomputing Center

Project repository: https://github.com/morousg/FusedKernelLibrary

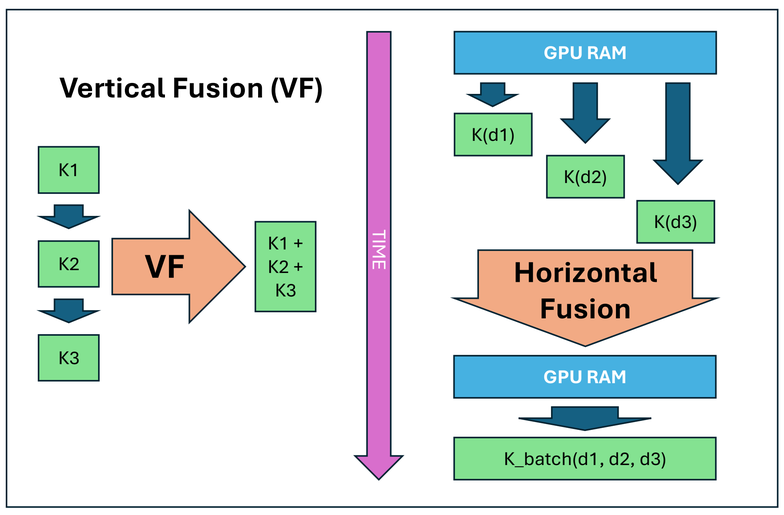
UNIVERSITAT POLITÈCNICA DE CATALUNYA BARCELONATECH
Escola de Doctorat

Barcelona Supercomputing Center
Centro Nacional de Supercomputación

ABSTRACT

Kernel fusion is a very popular optimization in GPU libraries, but the development is costly and code modularity and reusability is not obvious. The Fused Kernel Library (FKL) presents a solution to perform both Vertical Fusion (VF) and Horizontal Fusion (HF) in an automatic way, abstracting the complexities away from the final user, and only requiring a C++17 compatible compiler . In this poster, we present two new types of fusion that we implemented in the FKL, that extends the opportunities to apply fusion and automatically obtain faster kernels.

BACKGROUND



Vertical Fusion

```
K1::ParamsType params1{ /*Set params1*/ };
K2::ParamsType params2{ /*Set params2*/ };
K3::ParamsType params3{ /*Set params3*/ };

auto k1 = K1::build(params1);
auto k2 = K2::build(params2);
auto k3 = K3::build(params3);
executeOperations(input, output, stream, k1, k2, k3);
```

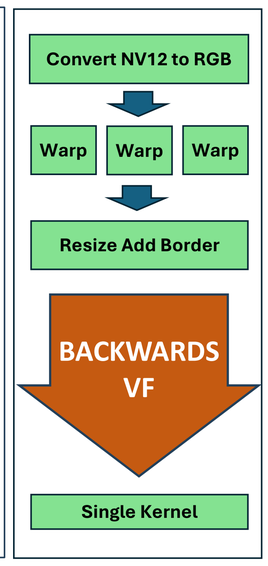
Horizontal Fusion

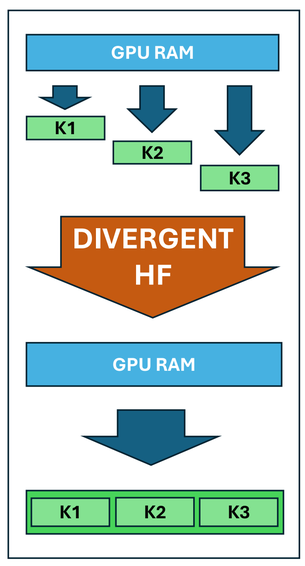
```
std::array<InputType, 50> input{ /*Set input pointers*/ };
std::array<K::ParamsType, 50> params{ /*Set params*/ };
std::array<OutputType, 50> output{ /*Set output pointers*/ };

executeOperations(input, output, stream, K::build(params));
```

CONTRIBUTION

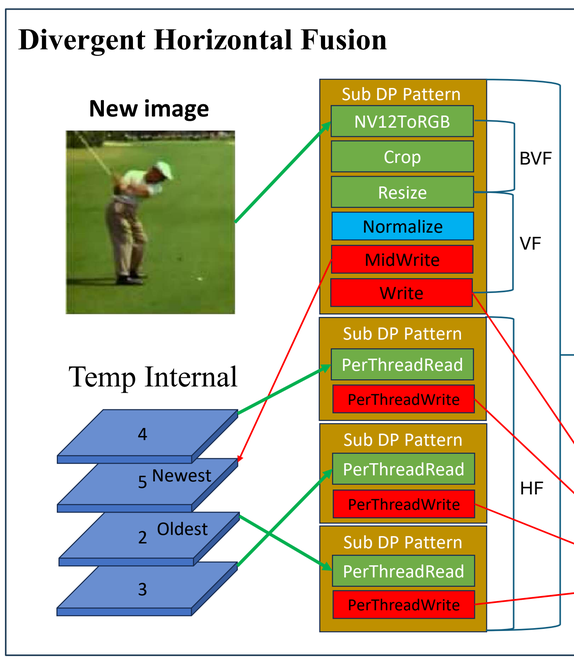
Backwards Vertical Fusion

```
std::array<Warp<IP::Linear>::ParamsType, 3>
warpParams{/*Set warp parameters*/ };

auto readOp = NV12ToRGB::build(input);
auto warpOp = Warp<IP::Linear>::build(warpParams);
auto resizeOp = Resize<RT::AddBorder>::build(targetSize);
auto normOp = Normalize::build(normParams);
auto writeOp = TensorWrite::build(output);

executeOperations(stream, readOp, warpOp, resizeOp,
normOp, writeOp);
```





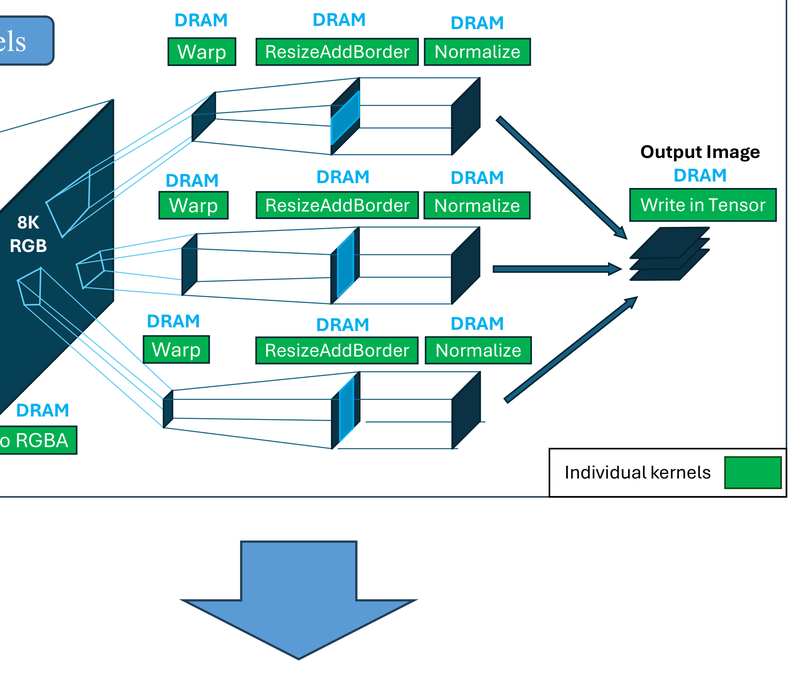
Divergent Horizontal Fusion

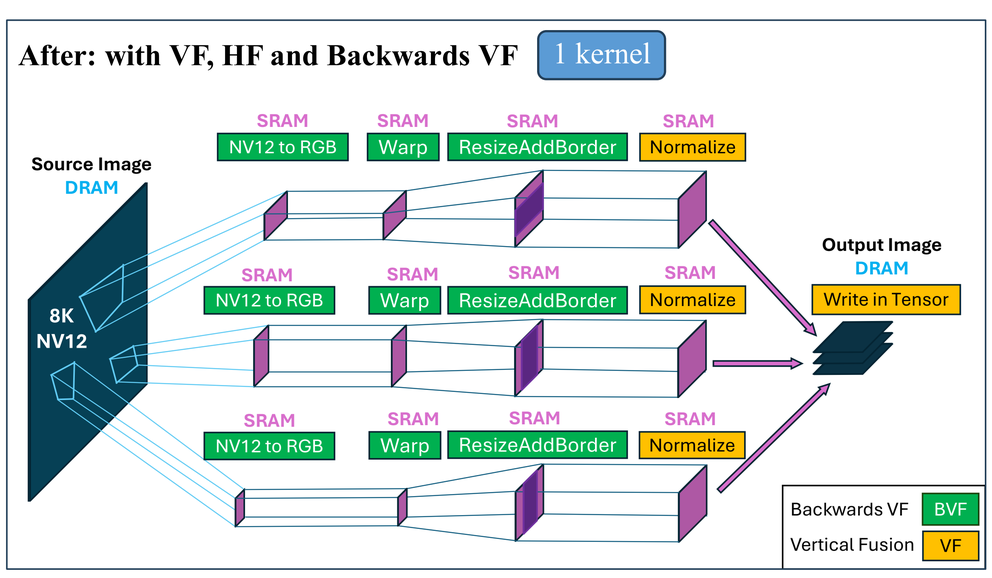


CircularTensor

```
int w = 64;
int h = 80 ;
int d = 4;
CircularTensor ct( w, h, d );

ct.update(stream,
        NV12ToRGB::build(newImage)
        Crop::build(Rect(12, 45, 45, 70)),
        Resize::build(Size(w, h)),
        Normalize::build(params));
```

Divergent Horizontal Fusion





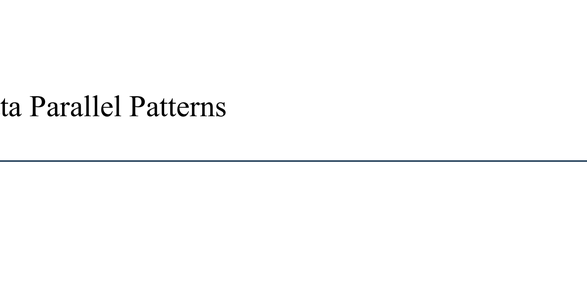
RESULTS

Backwards Vertical Fusion

1. **Image processing:**
   - 4x4K input images
   - NV12ToRGBA -> ColorBalance -> Undistort -> Stitch(1xFullHD) -> Overlay -> RGBAToNV12
   - **Speedup 10x** (RTX A2000 12GB)
2. **Computer Vision:**
   - 8 Warp crops over 1 or 2 source images
   - NV12ToRGB -> Warp -> ResizeAddBorder -> Normalize -> WriteTensor
   - **Speedup 35x** (RTX A2000 12GB)

- 50 crops across 4x4K images
- NV12ToRGB -> Crop -> Resize -> Normalize -> SplitTensor
- **Speedup 164x** (RTX A5000)

Divergent Horizontal Fusion



CircularTensor 15 images
28.4x speedup (RTX A5000)
0.9% of total time

CONCLUSIONS

BVF and DHF allow more complex fusion patterns with high level API's

Speedups are very relevant

Future work will include Reduction and SoftMax Data Parallel Patterns